SAEON Field Data System

Database Query Catalogue

What to build in api/src/db/ — Phase 1

Field	Detail
Database	**fds (PostgreSQL + PostGIS)**
Relates to	**Node API Developer Handover — read that first**
Date	**March 2026**
Purpose	**Defines every query the API needs. No SQL written here — query names and intent only.**

How to use this document

Each query listed here maps to a function you create inside api/src/db/queries.js (or split by domain).

Naming convention: verb + subject, camelCase. e.g. getStationById, createFieldVisit.

Each function receives named parameters and returns rows from the pg pool.

No raw SQL in routes — routes call query functions only. Keep queries in one place.

All timestamps come out of PostgreSQL as UTC. Format them in the API response layer.

1. Suggested File Structure

You can put all queries in one file or split by domain. For Phase 1 one file is fine:

```
api/src/db/
  pool.js          ← pg connection pool (already defined in handover doc)
  queries.js       ← all query functions for Phase 1
```

Each query function follows this pattern:

```
const pool = require('./pool');

async function getStationById(id) {
  const result = await pool.query( /* SQL here */ , [id]);
  return result.rows[0];
}

module.exports = { getStationById, ... };
```

2. Stations Queries

Used by GET /api/stations and GET /api/stations/:id

Query Name	Endpoint	Tables	Purpose
getAllStations	**GET /api/stations**	stations	Return all active stations. Include: id, name, display_name, data_family, region. Filter: active = true only.
getStationById	**GET /api/stations/:id**	stations	Return a single station by its id. Return null if not found.
getStationStreams	**GET /api/stations/:id**	station_data_streams	Return all data streams for a given station_id. Used to show what data types the station produces.
getStationLastVisit	**GET /api/stations/:id**	field_visits	Return the most recent field visit for a station. Used to show "last visited" on station detail.

3. Field Visit Queries

Used by POST /api/visits and GET /api/visits and GET /api/visits/:id

Query Name	Endpoint	Tables	Purpose
createFieldVisit	**POST /api/visits**	field_visits	Insert a new field visit. Parameters: station_id, technician_id, visited_at, notes. Return the new row.
getAllVisits	**GET /api/visits**	field_visits, stations, users	Return recent visits with station display_name and technician full_name joined in. Support optional station_id filter. Order by visited_at descending.
getVisitById	**GET /api/visits/:id**	field_visits, stations, users	Return a single visit with station and technician details joined. Return null if not found.
getVisitFiles	**GET /api/visits/:id**	uploaded_files	Return all files uploaded against a visit_id. Include: id, original_name, file_format, parse_status, date_range_start, date_range_end, record_count, uploaded_at.
getVisitReadings	**GET /api/visits/:id**	manual_readings	Return all manual readings recorded during a visit. Include: reading_type, value_numeric, value_text, unit, recorded_at, notes.
updateVisitStatus	**internal**	field_visits	Update the status of a visit (pending → complete or flagged). Called internally after all files in a visit are parsed.

4. File Upload Queries

Used by POST /api/visits/:id/files and GET /api/files/:id/download

Query Name	Endpoint	Tables	Purpose
getFileByHash	**POST /api/visits/:id/files**	uploaded_files	Look up an existing file by its SHA-256 hash before inserting. If found, return it — prevents duplicate uploads silently.
createUploadedFile	**POST /api/visits/:id/files**	uploaded_files	Insert a new file record. Parameters: visit_id, original_name, file_hash, file_size_bytes, storage_path, file_format. Set parse_status to pending. Return the new row with its id.
updateFileParsed	**internal (parser)**	uploaded_files	Update a file record after successful parsing. Set: parse_status = parsed, date_range_start, date_range_end, record_count.
updateFileParseError	**internal (parser)**	uploaded_files	Update a file record after a failed parse attempt. Set: parse_status = error, parse_error = error message.
getFileById	**GET /api/files/:id/download**	uploaded_files	Return a single file record by id. The route uses storage_path from this result to stream the file from disk.
getUnparsedFiles	**internal**	uploaded_files	Return all files where parse_status = pending. Used for background re-processing or an admin retry endpoint later.

5. Manual Readings Queries

Used by POST /api/visits/:id/readings

Query Name	Endpoint	Tables	Purpose
createManualReading	**POST /api/visits/:id/readings**	manual_readings	Insert a manual reading. Parameters: visit_id, reading_type, value_numeric, value_text, unit, recorded_at, notes. Return the new row.
getReadingsByVisit	**GET /api/visits/:id**	manual_readings	Return all readings for a given visit_id. Already covered by getVisitReadings — listed here for completeness.

6. Raw Measurements Queries

Used internally by parsers. Not directly exposed as endpoints in Phase 1.

Query Name	Endpoint	Tables	Purpose
bulkInsertMeasurements	**internal (parser)**	raw_measurements	Bulk insert parsed measurement rows for a file. Must handle the UNIQUE constraint on (stream_id, phenomenon_id, measured_at) — use ON CONFLICT DO NOTHING to skip duplicates silently.
getMeasurementsByStream	**future endpoint**	raw_measurements, phenomena	Return measurements for a stream_id between two timestamps. Include phenomenon name and unit joined from phenomena. Not needed in Phase 1 but name it now.
getMeasurementCount	**internal**	raw_measurements	Count measurements for a given file_id. Used to verify a parse completed correctly.

7. Supporting Lookup Queries

Small utility queries used across multiple routes.

Query Name	Endpoint	Tables	Purpose
getPhenomenonByName	**internal (parser)**	phenomena	Look up a phenomenon row by its name field. e.g. getPhenomenonByName("rainfall_tot"). Parsers need the phenomenon id before inserting measurements.
getAllPhenomena	**internal (parser)**	phenomena	Return all phenomena rows as a lookup map. Load once at parser startup rather than querying per row. Keyed by name for fast lookup.
getOrCreateStream	**internal (parser)**	station_data_streams	Find a stream by (station_id, stream_name). If it does not exist, insert it. Return the stream id. Needed because Campbell TOA5 files define their own table names.
getUserById	**internal (auth — Phase 2)**	users	Return a user row by id. Not needed in Phase 1 but define the function signature now so routes can call it when auth is added.
getUserByEmail	**internal (auth — Phase 2)**	users	Look up a user by email. Not needed in Phase 1 — define now.

8. Manager / Dashboard Queries

Not exposed as endpoints in Phase 1 but write them now — the manager dashboard in Phase 2 will use them immediately.

Query Name	Endpoint	Tables	Purpose
getRecentVisitsAllStations	**future: GET /api/dashboard**	field_visits, stations, users	Return the last N visits across all stations with station name and technician name. Marc's primary view.
getStationsNotVisitedSince	**future: GET /api/dashboard**	field_visits, stations	Return active stations that have had no field visit since a given date. Used to flag overdue stations.
getGroundwaterStationsMissingDipper	**future: GET /api/dashboard**	field_visits, manual_readings, stations	Return groundwater stations where the last visit has no dipper_depth reading recorded. Scientifically critical — flags visits where calibration data was not captured.
getFilesWithParseErrors	**future: GET /api/dashboard**	uploaded_files, field_visits, stations, users	Return all files where parse_status = error with station and technician context. Allows Marc to see what needs attention.
getStationDataCoverage	**future: GET /api/stations/:id/coverage**	uploaded_files, station_data_streams	Return the date range of all uploaded data for a station grouped by stream. Shows what time period of data SAEON holds.

9. Query Naming Convention

Follow this pattern consistently so the codebase stays readable:

Pattern	Example	When to use
get + Subject	**getStationById**	Fetch one row by identifier
getAll + Subject	**getAllStations**	Fetch multiple rows, possibly filtered
create + Subject	**createFieldVisit**	INSERT — return the new row
update + Subject + Field	**updateFileParsed**	UPDATE specific fields on a row
getOrCreate + Subject	**getOrCreateStream**	SELECT first, INSERT if not found
bulk + Verb + Subject	**bulkInsertMeasurements**	Batch INSERT multiple rows at once

One rule
Routes never contain SQL. Routes call query functions. Query functions never contain business logic. They only talk to the database.
Business logic (file hashing, format detection, parse orchestration) lives in routes or a separate services layer.

10. Full Query List at a Glance

All 23 query functions for Phase 1 + Phase 2 preparation:

#	Query Function	Phase	Domain
1	**getAllStations**	1	Stations
2	**getStationById**	1	Stations
3	**getStationStreams**	1	Stations
4	**getStationLastVisit**	1	Stations
5	**createFieldVisit**	1	Visits
6	**getAllVisits**	1	Visits
7	**getVisitById**	1	Visits
8	**getVisitFiles**	1	Visits
9	**getVisitReadings**	1	Visits
10	**updateVisitStatus**	1	Visits
11	**getFileByHash**	1	Files
12	**createUploadedFile**	1	Files
13	**updateFileParsed**	1	Files
14	**updateFileParseError**	1	Files
15	**getFileById**	1	Files
16	**getUnparsedFiles**	1	Files
17	**createManualReading**	1	Readings
18	**bulkInsertMeasurements**	1	Measurements
19	**getMeasurementsByStream**	1 (stub)	Measurements
20	**getMeasurementCount**	1	Measurements
21	**getPhenomenonByName**	1	Lookups
22	**getAllPhenomena**	1	Lookups
23	**getOrCreateStream**	1	Lookups
24	**getUserById**	2 prep	Users
25	**getUserByEmail**	2 prep	Users
26	**getRecentVisitsAllStations**	2 prep	Dashboard
27	**getStationsNotVisitedSince**	2 prep	Dashboard
28	**getGroundwaterStationsMissingDipper**	2 prep	Dashboard
29	**getFilesWithParseErrors**	2 prep	Dashboard
30	**getStationDataCoverage**	2 prep	Dashboard